

Molecular Dynamics - using critical and reduction

Because the archive containing the original sources was missing from the repository, the well-known EPCC MolDyn C program was rebuilt with the same layout (main.c -> domove, forces, mkekin, velavg, prnout).

Source files

main.c time-step driver loop (identical for serial and OpenMP)
md_util.c fcc, mxwell, domove, mkekin, velavg, prnout, dfill, dscal
forces.c serial force computation
forces_omp.c OpenMP force computation (the only file that changed)
Makefile make produces md_serial and md_omp;
 make CHUNK=n switches to schedule(static,n)

Build and execute

make # md_serial + md_omp
./md_serial
OMP_NUM_THREADS=4 ./md_omp
make CHUNK=10 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,10)

MolDyn - serial

```
$ make
```

```
$ ./md_serial
```

Molecular Dynamics Simulation example program

```
-----  
number of particles is ..... 13500  
side length of the box is ..... 25.323179  
cut off is ..... 3.750000  
reduced temperature is ..... 0.722000  
basic timestep is ..... 0.064000  
temperature scale interval ..... 10  
stop scaling at move ..... 20  
print interval ..... 5  
total no. of steps ..... 20  
OpenMP threads ..... 4
```

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 10.280000 s

MolDyn - parallel (2, 3, 4 threads)

```
$ OMP_NUM_THREADS=2 ./md_omp
```

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 10.274000 s

```
$ OMP_NUM_THREADS=3 ./md_omp
```

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 7.272000 s

```
$ OMP_NUM_THREADS=4 ./md_omp
```

move	k.e.	p.e.	tot.e.	temp.	pres.	vel.	rp
5	12619.1758	-91985.3542	-79366.1784	0.6232	-5.2880	0.1821	39.7
10	14619.4170	-86181.5919	-71562.1749	0.7220	-2.8265	0.1336	14.1
15	11405.1707	-82966.3254	-71561.1547	0.5633	-1.5094	0.1714	33.6
20	10825.0423	-82385.8646	-71560.8222	0.5346	-1.2219	0.1679	32.2

Time = 5.476000 s

```
MolDyn - schedule(static,n), 4 threads

$ make CHUNK=1 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,1)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.011000 s

$ make CHUNK=10 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,10)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.101000 s

$ make CHUNK=100 && OMP_NUM_THREADS=4 ./md_omp # schedule(static,100)
  move      k.e.      p.e.      tot.e.      temp.      pres.      vel.      rp
    5 12619.1758 -91985.3542 -79366.1784  0.6232 -5.2880  0.1821 39.7
   10 14619.4170 -86181.5919 -71562.1749  0.7220 -2.8265  0.1336 14.1
   15 11405.1707 -82966.3254 -71561.1547  0.5633 -1.5094  0.1714 33.6
   20 10825.0423 -82385.8646 -71560.8222  0.5346 -1.2219  0.1679 32.2

Time = 4.011000 s
```

Making forces() parallel

```
#pragma omp parallel for schedule(static) default(none) \
    shared(npart_, x, f, side, sideh, rcoffs) \
    private(j, xi,yi,zi, fxi,fyi,fzi, xx,yy,zz, rd, rrd, rrd2, \
        rrd3, rrd4, rrd6, rrd7, r148, forcex, forcey, forcez) \
    reduction(+:epot, vir)
for (i = 0; i < npart_*3; i += 3) {
    ...
    #pragma omp critical
```

```

    { f[j] -= forcex; f[j+1] -= forcey; f[j+2] -= forcez; }

    ...

    #pragma omp critical

    { f[i] += fxi; f[i+1] += fyi; f[i+2] += fzi; }

}

```

1. Private - every per-pair temporary (j, xi..zi, xx..zz, rd, rrd..rrd7, r148, forcex/y/z, fxi/fyi/fzi).
2. Reduction - the two accumulators epot (potential energy) and vir (virial). Each pair contributes to both, so every thread keeps its own partial sum and the sums are merged once the loop finishes.
3. Critical - the force array f[]. While thread A, working on particle i, subtracts forces from f[j] for all j > i, thread B may be adding to exactly the same element from its own row. To avoid lost updates these read-modify-write operations are enclosed in
#pragma omp critical.

Measurements

With 2, 3 and 4 threads every printed quantity matches the serial run exactly (the total energy of roughly -71 561 is conserved throughout).

version	threads	time (s)
serial	1	10.28
omp, static	2	10.27

omp, static	3	7.27
omp, static	4	5.48

Effect of schedule(static,n) with 4 threads

schedule	time (s)
static	5.48
static,1	4.01
static,10	4.10
static,100	4.01

The pair loop is triangular (j goes from $i+1$ to N), so under the plain static schedule thread 0 receives the most expensive rows. Using a small chunk size interleaves the rows across the threads and cuts the time by about 25 %. Past that point the exact chunk value hardly matters: what remains is the critical section in the inner loop, which serialises each force update and keeps the speed-up well short of the thread count.